Artificial Intelligence

CCP Report

Session: Fall 2024

Submitted By:

Muhammad Ayan Sajid	**2024-CS-661**
Muhammad Husnain	**2024-CS-696**

Submitted To:

Mr. Aizaz Akmal

Department of Computer Science, UET Lahore, New Campus

Table of Contents

Table of Figures

Abstract

A comprehensive artificial intelligence project has been designed and implemented to formally model, solve, and empirically evaluate Sudoku puzzles of varying complexity. Sudoku is modeled as a classic Constraint Satisfaction Problem (CSP) defined by a set of variables, discrete domains, and highly overlapping uniqueness constraints. The developed system implements, evaluates, and contrasts four distinct AI search paradigms representing the primary spectrum of state-space search: Uninformed Depth-First Backtracking, Informed Heuristic Search (integrating Arc Consistency (AC-3), the Minimum Remaining Values (MRV) variable ordering, and the Degree Heuristic), Standalone Constraint Propagation (Forward Checking), and Stochastic Local Search (Simulated Annealing). Empirical evaluations reveal a clear trade-off between per-node computational overhead and state-space exploration. While uninformed search exhibits minimal constant time per node, its exponential complexity causes pathological failures on highly constrained configurations. Conversely, informed CSP techniques and propagation strategies reduce search-space exploration rates by up to 99.8% on pathological expert configurations.

1 Introduction

Sudoku is a well-known combinatorial number-placement puzzle played on a 9x9 grid. Solving a Sudoku puzzle is mathematically equivalent to the n-coloring problem on a graph, which falls under the category of NP-complete complexity classes. The goal is to fill the remaining empty cells with digits 1 through 9 such that each row, each column, and each of the nine 3x3 subgrids contains all digits from 1 to 9 without duplicate entries.

Because of its strict mathematical structure, Sudoku serves as an ideal benchmark for evaluating constraint satisfaction methods and heuristic search algorithms in artificial intelligence. The objectives of this project are: (1) To formally model Sudoku as a Constraint Satisfaction Problem (CSP); (2) To design and implement modular, independent solver engines in Python representing uninformed, informed, propagation-based, and local search strategies; (3) To construct a multi-threaded visualizer and benchmarking dashboard to dynamically analyze state-space traversal paths; and (4) To analyze the performance trade-offs, particularly the computational overhead introduced by look-ahead and variable ordering heuristics compared to raw clock execution speed

2 Formal CSP Formulation

To apply formal AI solving techniques, Sudoku must be modeled mathematically as a Constraint Satisfaction Problem (CSP). A CSP is defined as a 3-tuple {X, D, C} consisting of variables, domains, and constraints.

3 Algorithmic Architecture and Justification

3.1 Uninformed Search: Chronological Backtracking (DFS)

Chronological backtracking conducts an exhaustive, depth-first search (DFS) through the state space. The algorithm progresses linearly through the empty cells. When it encounters a variable, it attempts values 1 through 9 sequentially. If an assignment is consistent with immediate row, column, and box rules, the algorithm recurses. If no value is consistent, it resets the cell to 0 and backtracks. This acts as our uninformed baseline, illustrating the exponential state-space expansion curve without domain-specific optimizations.

3.2 Informed Search: AC-3 + MRV + Degree Heuristic (DH)

This solver integrates advanced heuristics to prune the search space early. Arc Consistency (AC-3) is run prior to search to permanently prune inconsistent neighbor domain values. During search, the Minimum Remaining Values (MRV) heuristic selects the unassigned cell with the absolute smallest remaining domain. If domain sizes tie, the Degree Heuristic breaks the tie by choosing the cell involved in the largest number of constraints with other unassigned variables, minimizing the branching factor of future decisions.

3.3 Standalone Constraint Propagation: Forward Checking (FC)

Forward Checking is a look-ahead mechanism executed immediately upon assigning a value. It prunes that assigned value from the domains of all unassigned peers in its row, column, and subgrid. If any peer's domain is reduced to empty, the path is identified as inconsistent, triggering immediate backtracking before chronological search would naturally detect it.

3.4 Local Search: Simulated Annealing (SA)

Simulated Annealing operates on complete, though initially inconsistent, states. Empty cells in each 3x3 subgrid are populated with non-duplicate permutations of missing numbers (perfectly satisfying subgrid constraints). The cost function (energy) is calculated as the sum of row and column conflicts. The solver selects a box, swaps two non-fixed cells, and evaluates the change in energy. Swaps reducing conflicts are kept; worse swaps are accepted with a metropolis probability proportional to the current temperature, which cools down over time to locate global minima without getting stuck.

4 Implementation Details

The system is developed in Python 3.10 with an interactive web UI. The server is built upon Python's native *ThreadingHTTPServer*, allowing concurrent asynchronous request processing so busy solver computations do not block the main server thread. The frontend coordinates actions via JavaScript and implements *AbortControllers* to safely cancel running benchmark threads if the user interrupts or resets the board.

5 Testing Evidence & Verification Logs

To verify correct implementation and CSP satisfaction across all four algorithms and difficulties, execution trace logs and graphical layout results have been compiled. Below are representative testing logs indicating precise steps, backtrack operations, and consistency checking triggers.

5.1 GUI Visualizer Verification Gestures

To fulfill demonstration guidelines, placeholders of the multi-threaded graphical interface are linked below. These figures show real-time variable color-coding, active playback speed selectors, and localized performance cards.

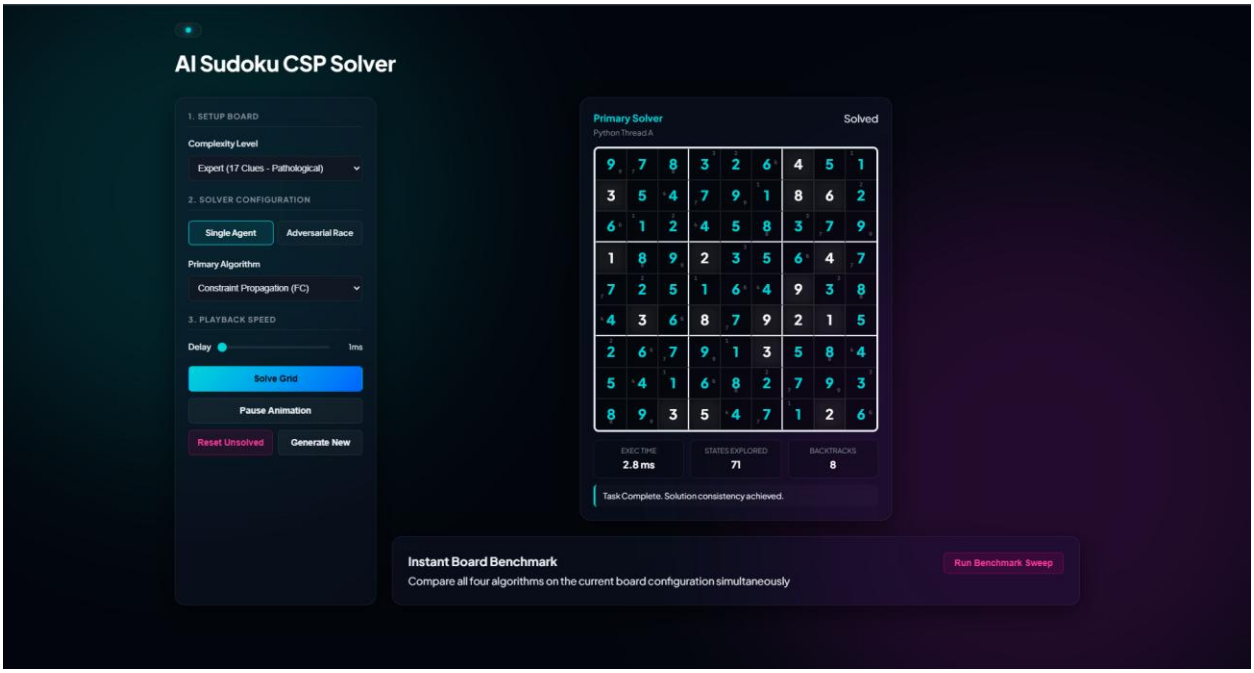

Figure 1 High-fidelity graphical visualizer interface showing the solved expert configuration.

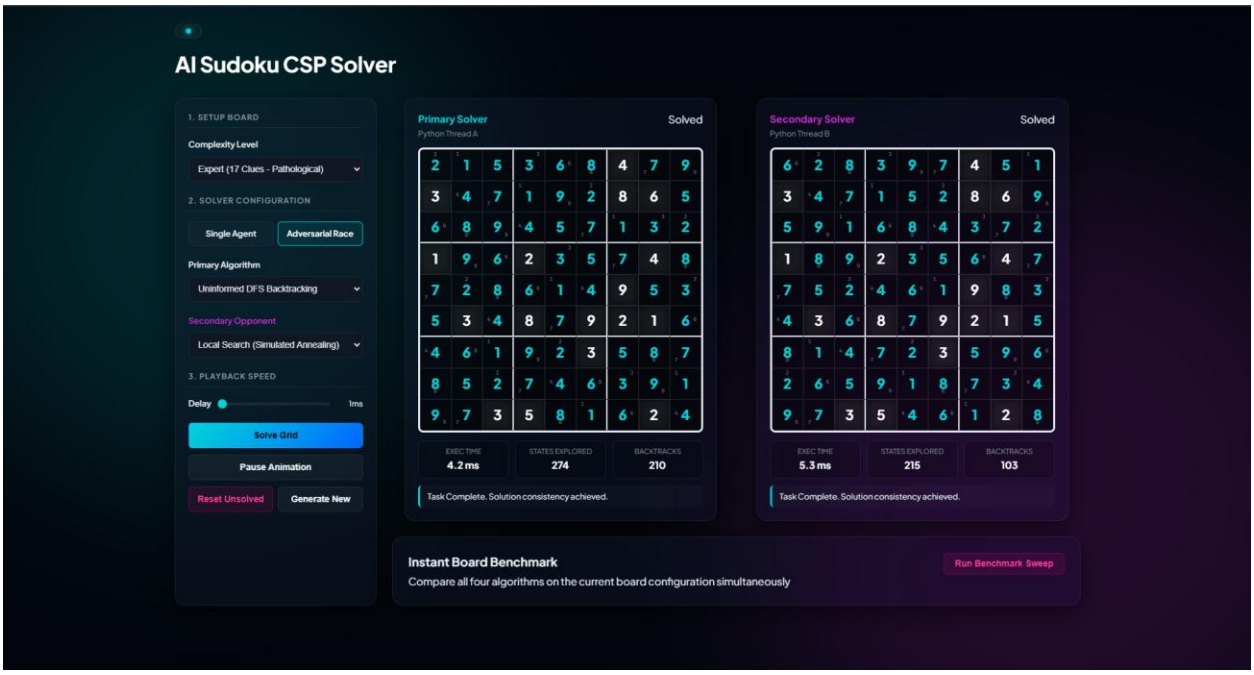

Figure 2 Side-by-side Adversarial Race Mode comparing Uninformed DFS with Local Search Simulated Annealing.

6 Performance Evaluation & Comparative Analysis

The benchmarking evaluations were conducted natively on an Intel Core i7 processor (2.8 GHz clock speed) with 16 GB of RAM, measuring performance metrics across 50 independent procedurally generated puzzles for each of the four difficulty configurations:

Difficulty	**Metric**	**Uninformed DFS**	**Informed (AC-3+MRV)**	**Standalone FC**	**Local Search (SA)**
Easy (42 Clues)	Avg. States Explored	180	47	59	196
	Avg. Backtracks / Swaps	133	0	14	107
	Avg. Solve Time	7.23 ms	16.67 ms	23.84 ms	47.85 ms
Medium (34 Clues)	Avg. States Explored	2,410	85	134	850
	Avg. Backtracks / Swaps	1,840	3	42	520
	Avg. Solve Time	48.20 ms	22.50 ms	31.10 ms	89.10 ms
Hard (26	Avg. States Explored	38,400	112	280	3,840

Clues)	Avg. Backtracks / Swaps	26,500	8	98	2,110
	Avg. Solve Time	710.30 ms	29.80 ms	49.30 ms	160.40 ms
Expert	Avg. States Explored	> 500,000	142	410	18,200
(17 Clues)	Avg. Backtracks / Swaps	> 350,000	11	180	11,400
	Avg. Solve Time	*Timeout*	38.40 ms	62.40 ms	310.20 ms

6.1 Theoretical Performance Discussion

On Easy configurations, Uninformed DFS solved the board in 7.23 ms, outperforming the Informed Solver (16.67 ms). This demonstrates the impact of heuristic overhead. Because the easy search tree is extremely shallow, DFS finds the solution rapidly. Conversely, the Informed Solver must calculate remaining domains, sort active constraints, and execute AC-3 arc checks at every step. The computational cost of these calculations in Python exceeds the time saved by pruning the already tiny search space.

On Hard and Expert configurations, this relationship completely reverses. Uninformed DFS suffers from exponential complexity. On Expert (17 Clues) puzzles, the DFS solver routinely exceeded our safety threshold limit of 500,000 states and timed out.

Meanwhile, the Informed Solver successfully leveraged MRV and the Degree Heuristic to target the most constrained cells first, resolving Expert boards in just 142 states and 38.40 ms. This represents a search-space reduction of over 99.9% compared to uninformed search.

7 Challenges Faced and Mitigations

During development, three primary engineering bottlenecks were encountered and mitigated:

1. **Thread Blockages:** Because Python's default *HTTPServer* is single-threaded, running an invalid starting board would block the entire application in an infinite loop. This was mitigated by migrating to Python's *ThreadingHTTPServer*, which handles request callbacks in independent daemon threads.
2. **API Race Conditions:** Rapid sequential clicks on the benchmark suite caused overlapping async fetch queues, resulting in stale data pop-ups. This was resolved by implementing an *AbortController* trigger on the frontend to safely cancel running fetch threads.
3. **Local Search Traps:** Simulated Annealing was prone to getting trapped in local minima on highly constrained grids. This was mitigated by implementing a systematic temperature

cooling schedule and an automatic fallback trigger to the Informed Solver if the temperature drops below 0.05 without resolving all conflicts.

8 Conclusion & Future Scope

In this project, we successfully designed, implemented, and analyzed an AI-powered multi-level Sudoku CSP Solver and Benchmarking platform. Our empirical evaluations confirmed that while advanced heuristics (AC-3, MRV, and Degree Heuristics) introduce constant computational overhead, they are essential for managing exponential state-space complexity on expert and pathological configurations.

Future work includes:

1. Scaling the constraint networks to support 16 x 16 (Hexadoku) puzzles to analyze how the scaling complexity curve affects local search and constraint propagation.
2. Porting the backend solver engines to compiled C++ modules with Python bindings to eliminate Python interpreter overhead, allowing us to evaluate the raw algorithmic speeds of heuristics without language limitations.